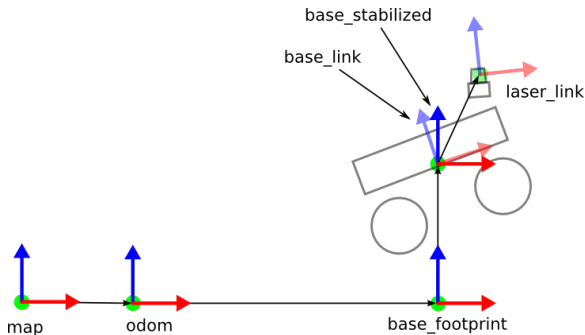


Introduction to ROS2: Basics, Motion, and Vision

Geesara Kulathunga

What is TF2 in ROS 2?

- TF2 is a framework to *track transforms between different coordinate frames* over time.
- Allows components to transform points, vectors, or poses from one frame to another.
- Enables components to remain decoupled and reuse code across different robots.



Static vs Dynamic Transforms



- **Static:** Fixed transforms that do not change over time, such as the transform between a robot's base frame and a fixed sensor like a camera. These transforms are broadcast once and assumed constant throughout the operation. They are typically published using `tf2_ros::StaticTransformBroadcaster`.
- **Dynamic:** Transforms that change over time, representing moving parts or components, such as the transform between the robot base and a moving wrist or end-effector. These are continuously updated and broadcast, usually with `tf2_ros::TransformBroadcaster`, to reflect the current state of the robot.
- **Use Cases:** Static transforms are ideal for components with fixed spatial relationships, simplifying the transform tree and reducing computational overhead. Dynamic transforms enable accurate real-time tracking of articulated parts or sensors mounted on moving joints.

Static vs Dynamic Transforms



- **Performance:** Broadcasting only static transforms once reduces network traffic and computational load, while dynamic transforms require frequent publishing at a fixed rate (e.g., 30Hz) to maintain accurate pose information.
- **Examples:**
 - Static: The offset from the robot base to a LIDAR sensor permanently mounted on the chassis.
 - Dynamic: The changing pose of a robotic arms end-effector as it moves.

```
static_transform_broadcaster->sendTransform(base_to_camera);  
dynamic_transform_broadcaster->sendTransform(base_to_wrist);
```

Mathematical Background



$$T = \begin{pmatrix} R & \vec{t} \\ \mathbf{0}^\top & 1 \end{pmatrix},$$

where R is a 3×3 rotation matrix and $\vec{t}$ is a 3D translation vector.

Example: Consider a rotation of 90 degrees around the z-axis and a translation of $\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$.

Mathematical Background

Example: Consider a rotation of 90 degrees around the z-axis and a translation of $\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$.

Then,

$$R = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad \vec{t} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}.$$

The homogeneous transformation matrix T is

$$T = \begin{pmatrix} 0 & -1 & 0 & 1 \\ 1 & 0 & 0 & 2 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Rotation from Roll-Pitch-Yaw

Rotation is often defined by roll Φ , pitch Θ , and yaw Ψ :

$$R = R_z(\Psi)R_y(\Theta)R_x(\Phi),$$

with:

$$R_x(\Phi) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \Phi & -\sin \Phi \\ 0 & \sin \Phi & \cos \Phi \end{pmatrix}$$

$$R_y(\Theta) = \begin{pmatrix} \cos \Theta & 0 & \sin \Theta \\ 0 & 1 & 0 \\ -\sin \Theta & 0 & \cos \Theta \end{pmatrix}$$

$$R_z(\Psi) = \begin{pmatrix} \cos \Psi & -\sin \Psi & 0 \\ \sin \Psi & \cos \Psi & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Why Use Roll, Pitch, and Yaw?



Rotation in 3D space is often described using **roll** (Φ), **pitch** (Θ), and **yaw** (Ψ) angles because:

- **Intuitive Interpretation:**

- **Roll** (Φ): rotation about the x -axis (tilting side-to-side).
- **Pitch** (Θ): rotation about the y -axis (tilting forward/backward).
- **Yaw** (Ψ): rotation about the z -axis (turning left/right).

- **Sequential Rotations:** These angles represent a sequence of rotations about fixed axes (commonly ZYX or XYZ order), which is easy to understand and apply in robotic control.
- **Compact Representation:** Only three parameters are needed to represent rotation, compared to nine values in a 3×3 matrix.
- **Sensor Compatibility:** IMUs and gyroscopes commonly provide orientation data as RPY angles.
- **Widespread Usage:** Many robotics frameworks (e.g., ROS TF) and simulation tools use RPY for ease of integration and human readability.

Gimbal Lock Example

Gimbal lock occurs when two of the three rotation axes align, causing a loss of one degree of freedom in rotation (https://www.youtube.com/watch?v=_Mdq5EhJ56k).

Example: Using roll-pitch-yaw (ZYX Euler angles), when pitch $\Theta = \pm 90^\circ$, the roll and yaw axes align:

$$R = R_z(\Psi) R_y\left(\pm \frac{\pi}{2}\right) R_x(\Phi)$$

This causes the system to lose the ability to distinguish between roll and yaw both affect rotation around the same axis.

Effect: The robot or camera may exhibit erratic behavior or sudden jumps in orientation.

To avoid this, we use **quaternions**, which provide a singularity-free representation of rotation.

Using Quaternions in TF2

$$q = w + xi + yj + zk,$$

To avoid gimbal lock, we often use quaternions.

$$q = \left(\cos \frac{\theta}{2}, \vec{u} \sin \frac{\theta}{2} \right),$$

Where $\vec{u}$ is a unit vector representing the axis of rotation, and θ is the rotation angle.

Quaternions allow smooth interpolation (slerp) and avoid singularities inherent in Euler angles.

Rotation Example in C++



```
tf2::Quaternion q; q.setRPY(0, 0, M_PI/2); transform.transform.rotation = tf2::toMsg(q);
```

TF2: Understanding



In ROS2 TF2, the function call

```
buffer.lookupTransform(target_frame, source_frame, time);
```

returns the transform that converts data **from the source frame to the target frame**.

Meaning:

- `target_frame`: The frame **you want the data in**.
- `source_frame`: The frame **you are transforming from**.

It returns: the transform $T_{\text{source}}^{\text{target}}$ such that a point $\vec{p}_{\text{source}}$ in the source frame is transformed into the target frame as:

$$\vec{p}_{\text{target}} = T_{\text{source}}^{\text{target}} \cdot \vec{p}_{\text{source}}$$

Transform Chaining

To find T_{AC} given T_{AB} and T_{BC} ,

$$T_{AC} = T_{AB}T_{BC},$$

This obeys non-commutativity:

$$T_{AB}T_{BC} \neq T_{BC}T_{AB}.$$

How to determine the camera's position in the base frame?

Practical Example

$$T_{base \rightarrow wrist} \times T_{wrist \rightarrow camera} = T_{base \rightarrow camera}.$$

This lets you resolve transforms to find where the camera is in base frame.

```
geometry_msgs::msg::TransformStamped base_to_camera =  
buffer.lookupTransform("base_link", "camera", rclcpp::Time(0));
```

Summary



- TF2 converts between multiple Cartesian frames efficiently.
- Static vs Dynamic transforms.
- Operations obey homogenous transform rules.
- Quaternions elegantly handle 3D rotations.
- Chaining transforms lets you resolve positions across numerous links.